

12.3.2 数据流体系结构风格

数据流体系结构是一种计算机体系结构，直接与传统的冯·诺依曼体系结构或控制流体系结构进行了对比。数据流体系结构没有概念上的程序计数器：指令的可执行性和执行仅基于指令输入参数的可用性来确定，因此，指令执行的顺序是不可预测的，即行为是不确定的。数据流体系结构风格主要包括批处理风格和管道-过滤器风格。

1. 批处理体系结构风格

在批处理风格（如图12-3所示）的软件体系结构中，每个处理步骤是一个单独的程序，每一步必须在前一步结束后才能开始，并且数据必须是完整的，以整体的方式传递。它的基本构件是独立的应用程序，连接件是某种类型的媒介。连接件定义了相应的数据流图，表达拓扑结构。

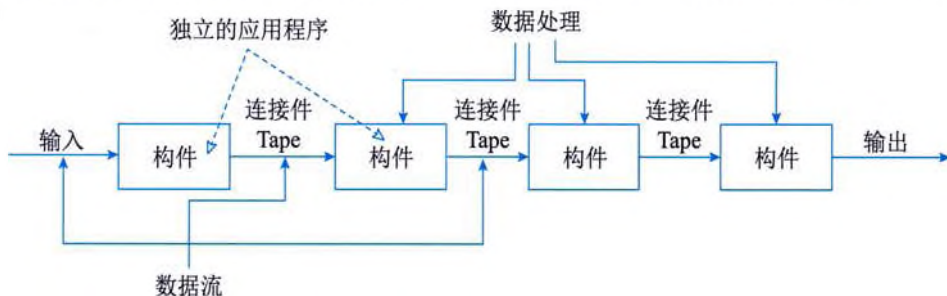


图 12-3 批处理体系结构风格

2. 管道-过滤器体系结构风格

当数据源源不断地产生，系统就需要对这些数据进行若干处理（分析、计算、转换等）。现有的解决方案是把系统分解为几个序贯的处理步骤，这些步骤之间通过数据流连接，一个步骤的输出是另一个步骤的输入。每个处理步骤由一个过滤器（Filter）实现，处理步骤之间的数据传输由管道（Pipe）负责。每个处理步骤（过滤器）都有一组输入和输出，过滤器从管道中读取输入的数据流，经过内部处理，然后产生输出数据流并写入管道中。因此，管道-过滤器体系结构风格（如图12-4所示）的基本构件是过滤器，连接件是数据流传输管道，将一个过滤器的输出传到另一过滤器的输入。

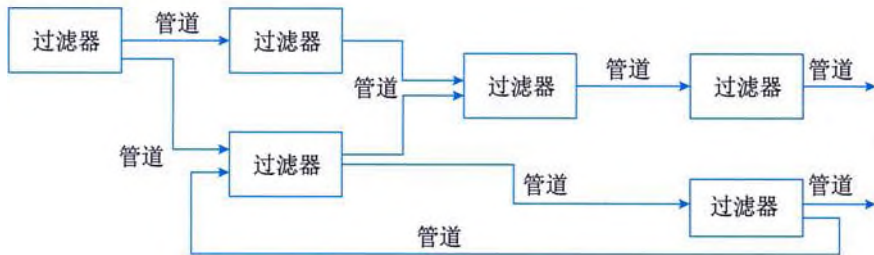


图 12-4 管道-过滤器体系结构风格

12.3.3 调用/返回体系结构风格

调用返回体系结构风格是指在系统中采用了调用与返回机制。利用调用-返回实际上是一

种分而治之的策略，其主要思想是将一个复杂的大系统分解为若干子系统，以便降低复杂度，并且增加可修改性。程序从其执行起点开始执行该构件的代码，程序执行结束，将控制返回给程序调用构件。调用/返回体系结构风格主要包括主程序/子程序风格、面向对象风格、层次型风格以及客户端/服务器风格。

1. 主程序/子程序风格

主程序/子程序风格一般采用单线程控制，把问题划分为若干处理步骤，构件即为主程序和子程序。子程序通常可合成为模块。过程调用作为交互机制，即充当连接件。调用关系具有层次性，其语义逻辑表现为子程序的正确性取决于它调用的子程序的正确性。

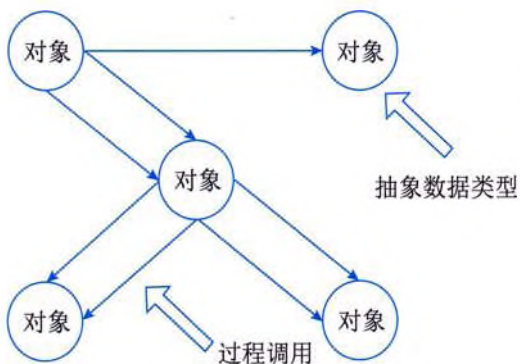


图 12-5 面向对象体系结构风格

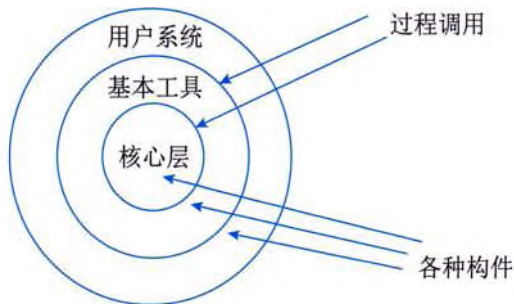


图 12-6 层次型体系结构风格

2. 面向对象体系结构风格

抽象数据类型概念对软件系统有着重要作用，目前软件界已普遍转向使用面向对象系统。这种风格建立在数据抽象和面向对象的基础上，数据的表示方法和它们的相应操作封装在一个抽象数据类型或对象中。这种风格的构件是对象，或者说是抽象数据类型的实例（如图 12-5 所示）。

3. 层次型体系结构风格

层次系统（如图 12-6 所示）的组成为一个层次结构，每一层为上层提供服务，并作为下层的客户。在一些层次系统中，除了一些精心挑选的输出函数外，内部的层接口只对相邻的层可见。在这样的系统中，构件在层上实现了虚拟机。连接件由决定层间如何交互的协议来定义，拓扑约束包括对相邻层间交互的约束。由于每一层最多只影响两层，同时只要给相邻层提供相同的接口，允许每层用不同的方法实现，这同样为软件重用提供了强大的支持。

4. 客户端/服务器体系结构风格

在 IT 发展过程中，网络计算经历了从集中式计算模型到分布式计算模型的演变。在集中式计算技术时代，广泛使用的是大型机（或小型机）计算模型。它是通过一台物理上与宿主机相连接的非智能终端来实现宿主机上的应用程序。在多用户环境中，宿主机应用程序既负责与用户的交互，又负责对数据的管理。集中式的系统使用户能共享贵重的硬件设备，例如，磁盘机、打印机和调制解调器等。但随着用户的增多，对宿主机能力的要求增高，而且开发人员必须为每个新的应用重新设计同样的数据管理构件。

20 世纪 80 年代以后，集中式结构逐渐被以 PC 为主的微机网络所取代。PC 和工作站的采

用，永远改变了协作计算模型，导致了分布式计算模型的产生。一方面，由于大型机系统固有的缺陷（例如，缺乏灵活性），无法适应信息量急剧增长的需求，并为整个企业提供全面的解决方案；另一方面，由于微处理器的日新月异，其强大的处理能力和低廉的价格使微机网络迅速发展，用户可以选择适合自己需要的工作站、操作系统和应用程序。

1) 二层架构

客户机/服务器（Client/Server, C/S）架构是基于资源不对等，且为实现共享而提出来的，是20世纪90年代成熟起来的技术，C/S架构定义了工作站（客户应用程序）如何与服务器相连，以实现数据和应用分布到多台计算机上。服务器负责有效地管理系统的资源，其主要任务集中于对DBMS的管理和控制，以及数据的备份与恢复；客户应用程序的主要任务是提供用户与数据库交互的界面，向服务器提交用户请求并接收来自服务器的信息，对存在于客户端的数据执行应用逻辑要求。这是一种“胖客户机（fat client）、瘦服务器（thin server）”的架构，其处理流程如图12-7所示。

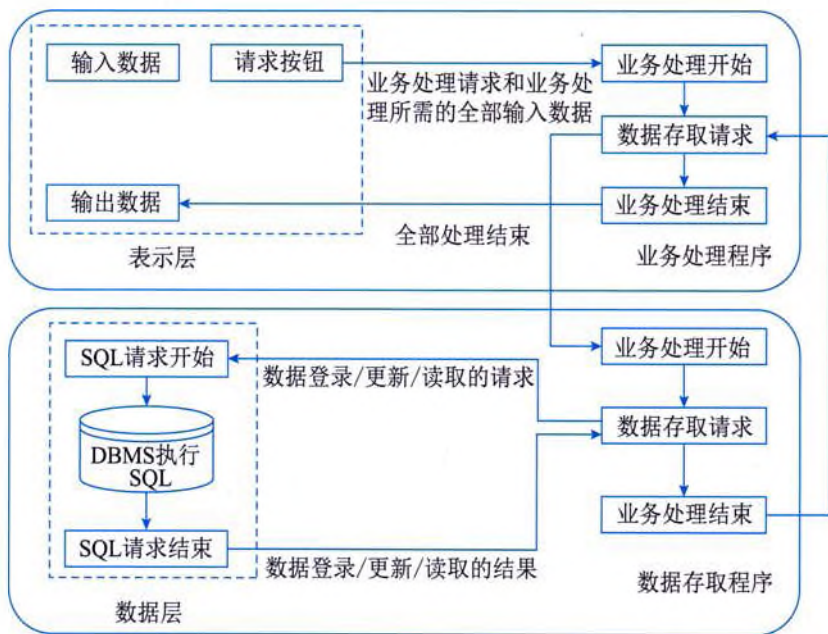


图 12-7 C/S 架构的一般处理流程

与集中式系统相比，C/S架构的优点主要在于，系统的客户应用程序和服务构件分别运行在不同的计算机上，系统中每台服务器都可以适合各构件的要求，这对于硬件和软件的变化显示出极大的适应性和灵活性，而且易于对系统进行扩充和缩小。在C/S架构中，系统中的功能构件充分隔离，客户应用程序的开发集中于数据的显示和分析，而服务器的开发则集中于数据的管理，不必在每一个新的应用程序中都要对一个DBMS进行编码。将大的应用处理任务分布到许多通过网络连接的低成本计算机上，以节约大量费用。

C/S架构具有强大的数据操作和事务处理能力，模型思想简单，易被人们理解和接受。但随着企业规模的日益扩大，软件的复杂程度不断提高，C/S架构逐渐暴露出以下缺点：

(1) 开发成本较高。C/S 架构对客户端软硬件配置要求较高，尤其是软件的不不断升级，对硬件要求不断提高，增加了整个系统的成本。

(2) 客户端程序设计复杂。采用 C/S 架构进行软件开发，大部分工作量放在客户端的程序设计上，客户端显得十分庞大。

(3) 用户界面风格不一，使用繁杂，不利于推广使用。

(4) 软件移植困难。采用不同开发工具或平台开发的软件，一般互不兼容，不能或很难移植到其他平台上运行。

(5) 软件维护和升级困难。采用 C/S 架构的软件要升级，开发人员必须到现场为客户机升级，每个客户机上的软件都需要维护。对软件的一个小小改动，每一个客户端都必须更新。

(6) 新技术不能轻易应用。因为一个软件平台及开发工具一旦选定，不可能轻易更改。

(7) 可扩展性差。C/S 架构是单一服务器且以局域网为中心的，所以难以扩展至大型企业广域网或 Internet，软硬件的组合和集成能力有限。客户机的负荷太重，难以管理大量的客户机，系统的性能容易变坏。

(8) 系统安全性难以保证。因为客户端程序可以直接访问数据库服务器，所以，在客户端计算机上的其他程序也可想办法访问数据库服务器，从而使数据库的安全性受到威胁。

正是因为 C/S 架构有这么多缺点，因此，三层 C/S 架构应运而生。为了区分，把传统的 C/S 架构称为二层 C/S 架构。

2) 三层 C/S 架构

与二层 C/S 架构相比，在三层 C/S 架构中，增加了一个应用服务器。可以将整个应用逻辑驻留在应用服务器上，而只有表示层存在于客户机上。这种客户机称为瘦客户机 (thin client)。三层 C/S 架构将应用系统分成表示层、功能层和数据层三个部分，如图 12-8 所示。

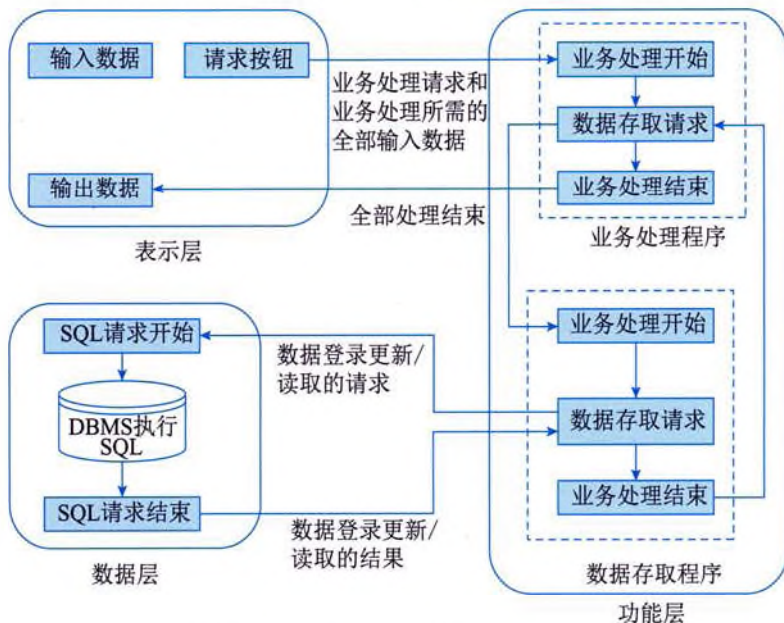


图 12-8 三层 C/S 架构的一般处理流程

(1) 表示层。表示层是系统的用户接口部分，担负着用户与系统之间的对话功能。它用于检查用户从键盘等输入的数据，显示输出的数据。为使用户能直观地进行操作，通常使用图形用户界面，操作简单、易学易用。在变更用户界面时，只需改写显示控制和数据检查程序，而不影响其他两层。检查的内容也只限于数据的形式和取值的范围，不包括有关业务本身的处理逻辑。

(2) 功能层。功能层也称为业务逻辑层，是将具体的业务处理逻辑编入程序中。例如，在制作订购合同时计算合同金额、按照预定的格式配置数据、打印订购合同，而处理所需的数据则要从表示层或数据层取得。

(3) 数据层。数据层相当于二层 C/S 架构中的服务器，负责对 DBMS 进行管理和控制。

三层 C/S 架构对这三层进行明确分割，并在逻辑上使其独立。在二层 C/S 架构中，数据层作为 DBMS 已经独立出来，所以，三层 C/S 架构的关键是要将表示层和功能层分离成各自独立的程序，并且还要使这两层间的接口简洁明了。通常的做法是只将表示层配置在客户机中，如图 12-9 (a) 或图 12-9 (b) 所示。如果像图 12-9 (c) 所示的那样连功能层也放在客户机中，与二层 C/S 架构相比，其程序的可维护性要好得多，但是其他问题并未得到解决。客户机的负荷太重，其业务处理所需的数据要从服务器传给客户机，所以系统的性能容易变坏。

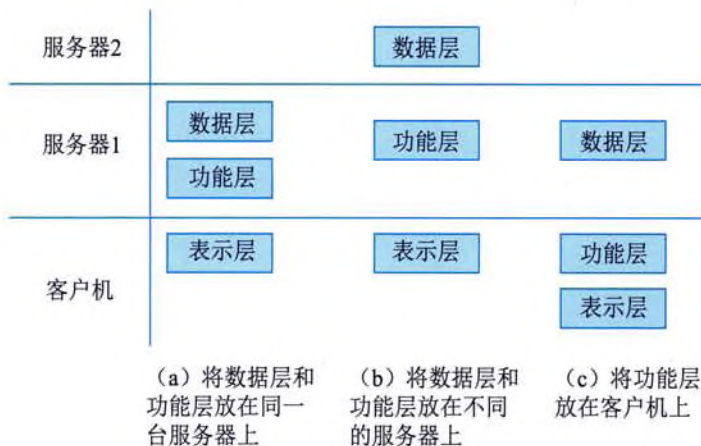


图 12-9 三层 C/S 架构的物理部署

如果将功能层和数据层分别放在不同的服务器中，如图 12-9 (b) 所示，则服务器之间也要进行数据传送。由于三层是分别放在各自不同的硬件系统上的，所以灵活性很高，能够适应客户机数目的增加和处理负荷的变动。例如，在追加新业务处理时，可以相应增加装载功能层的服务器（应用服务器）。因此，系统规模越大，这种形态的优点就越显著。在三层 C/S 架构中，中间件是最重要的构件。

与传统的二层 C/S 架构相比，三层 C/S 架构具有以下优点：

(1) 允许合理地划分三层的功能，使之在逻辑上保持相对独立性，从而使整个系统的逻辑结构更为清晰，能提高系统的可维护性和可扩展性。

(2) 允许更灵活、有效地选用相应的平台和硬件系统，使之在处理负荷能力上与处理特性上分别适应于结构清晰的三层，并且这些平台和各个组成部分可以具有良好的可升级性和开放性。

(3) 系统的各层可以并行开发，各层也可以选择各自最适合的开发语言，使之能并行且高